

Przygotowanie do rozmowy Ideo Software – Mobile Developer (Flutter)

Adrian Rojek
Spotkanie: 11.09.2026, godz. 10:00
ul. Nad Przyrwą 13, Rzeszów
Uczestnicy: Jan Mazur, Bohdan Sushchak

Spis treści

1	Werdykt: czy będzie live coding?	2
2	Pitch 30 sekund	2
3	Soft / HR / praca w software house	2
4	Flutter i Dart – fundamenty	3
5	Zarządzanie stanem (Provider, Cubit, Bloc)	5
6	REST API i komunikacja klient–serwer	5
7	Firebase	6
8	UI, design i jakość wizualna	7
9	Natywne Android / iOS i mosty	7
10	Publikacja App Store / Google Play	8
11	Architektura, czysty kod, git	8
12	Pytania o Twoje projekty	9
12.1	DEVEJI – Flutter komercyjnie	9
12.2	Fruit & Vegetable Tracker (Flutter, prywatne)	9
12.3	Water Reminder (SwiftUI)	9
12.4	Fitness Tracker (SwiftUI)	10
12.5	MacSpaceCleaner	10
12.6	Courier Task Management (Electron, prywatne)	10
12.7	Bus Timetable (Laravel)	10
12.8	Inne	11
13	Pułapki i uczciwe odpowiedzi „co bym poprawił”	11
14	Pytania behawioralne (STAR – szkielety)	11
15	Mini zadania kodowe / whiteboard (na wszelki wypadek)	12
16	Android / iOS – pytania pogłębiające	12

17 Bazy lokalne i offline	13
18 Bezpieczeństwo mobile	13
19 Wydajność i stabilność	13
20 Proces, Agile, współpraca z designem	14
21 Pytania o Ideo i technologię firmy	14
22 Twoje pytania do Ideo (zadaj 3–5)	14
23 Checklista dzień przed / rano	14

1 Werdykt: czy będzie live coding?

Raczej nie pełny live coding na tym spotkaniu – niskie/średnie ryzyko krótkiego zadania „na sucho”.

- Spotkanie **max 45 min**, stacjonarnie, z liderem wdrożeń i mobile developerem – klasyczny pierwszy etap: kultura + CV + pytania techniczne.
- W 45 minutach zwykle: intro/firma ~10 min, projekty/CV ~15–20, Flutter/Firebase/REST ~10–15, Twoje pytania ~5. Mało miejsca na spokojne kodowanie w IDE.
- Software house’y w PL na pierwszym spotkaniu częściej robią rozmowę techniczną / „jak byś to zrobił”, a zadanie kodowe/take-home zostawiają na później.
- Oferta celuje w doświadczenie projektowe (store, Firebase, UI, native), nie w algorytmy.

Na wszelki wypadek przygotuj: krótkie zadanie ustne (widget tree, `FutureBuilder` vs `Cubit`, `JSON`→`model`, `ListView.builder`, `REST`), opis architektury Fruit Tracker / DEVEJI. Następna runda może mieć live coding lub zadanie domowe.

2 Pitch 30 sekund

P: Opowiedz coś o sobie / kim jesteś jako developer?

O: Jestem studentem informatyki na Uniwersytecie Rzeszowskim (II stopień). Specjalizuję się w mobile – SwiftUI na iOS oraz Flutter cross-platform. W DEVEJI robiłem aplikacje Flutter z Firebase i publikacją w App Store i Google Play. Prywatnie mam apkę Flutter do śledzenia owoców/warzyw z ML i SQLite, a publicznie m.in. Water Reminder i Fitness Tracker w SwiftUI oraz MacSpaceCleaner pokazujący świadome podejście do architektury.

P: Dlaczego Ideo / dlaczego ta oferta?

O: Szukacie Flutterowca z natywnym zapleczem, Firebase, REST i aplikacjami w sklepach – to dokładnie mój stack. Chcę wejść w software house z realnymi klientami z wielu branż, uczyć się od doświadczonego zespołu i rozwijać jakość UI oraz architektury. Rzeszów + hybryda też mi pasuje.

3 Soft / HR / praca w software house

P: Jak wyglądała Twoja współpraca w zespole w DEVEJI?

O: Pracowałem nad aplikacjami Flutter iOS/Android, backend Firebase, wdrożenia do sklepów. Współpracowałem przy wymaganiach, implementacji UI, integracji API i poprawkach po feedbacku. Uczę się komunikować ryzyka wcześniej (np. limity platform, review store).

P: Jak radzisz sobie z niejasnymi wymaganiami klienta?

O: Dopytuję o happy path i edge case’y, proponuję najprostsze MVP, potwierdzam założenia na piśmie/ticketach, robię małe iteracje zamiast wielkiego big-bang. Jeśli coś jest niejasne w designie – pytam designera/PM zanim zgadnę.

P: Co robisz, gdy utkniesz na bugach?

O: Reprodukuję minimalnie, sprawdzam logi (Flutter DevTools, Crashlytics), bisect zmian, czytam dokumentację/plugin issues, izoluję warstwę (UI vs state vs sieć). Jeśli blokuję sprint – eskaluję wcześniej z kontekstem i planem B.

P: Preferujesz remote, hybrydę czy biuro?

O: Hybryda według potrzeb – jak w ofercie. Na start chętnie bywam w biurze (onboarding, pairing), potem elastycznie pod projekt.

P: Jakie masz oczekiwania wobec onboardingu?

O: Dostęp do repo, standardów code review, stacku stanu (Bloc/Cubit/Provider), procesu release i kontaktu do mentora. Chcę szybko dostarczać małe PR-y i uczyć się konwencji zespołu.

P: Gdzie widzisz się za rok?

O: Jako samodzielny Flutter developer w Ideo: pewnie prowadzę feature'y end-to-end, znam natywne mosty, dbam o jakość UI i performance, pomagam juniorom jeśli będzie okazja.

P: Ile zarabiałeś / jakie masz oczekiwania finansowe?

O: Przygotuj widełki rynkowe junior/mid Flutter PL (B2B/UoP). Powiedz, że jesteś elastyczny względem formy (UoP/B2B) i zakresu obowiązków; dopytaj o widełki Ideo jeśli nie chcesz strzelać pierwszy.

P: Czy masz aplikacje w sklepach?

O: Tak – w DEVEJI uczestniczyłem w publikacji na App Store i Google Play. Mam też własne projekty iOS (m.in. Water Reminder, Fitness Tracker). Fruit Tracker (Flutter) mogę omówić architektonicznie; status publikacji doprecyzuj zgodnie z prawdą.

P: Czy możesz pracować też przy innych technologiach mobilnych?

O: Tak – oferta to przewiduje. Mam SwiftUI/iOS, znam Kotlin na poziomie podstawowym, Electron/Node przy projekcie kurierskim, web przy NEW PORTABLE DEVICES. Flutter będzie core, ale nie boję się kontekstów natywnych i backendowych.

P: Jak łączysz studia z pracą?

O: Magisterka na UR od 02.2026; wcześniej łączyłem studia z DEVEJI/G2A. Priorytetyzuję deliverables, komunikuję urlopy sesyjne z wyprzedzeniem.

4 Flutter i Dart – fundamenty

P: Czym jest Flutter i jak działa pod spodem?

O: UI toolkit od Google: piszesz w Dart, silnik (Impeller/Skia) rysuje UI. Jedna baza kodu na iOS/Android (oraz web/desktop). Widżety to niemutowalne konfiguracje drzewa UI; stan trzymasz osobno.

P: StatelessWidget vs StatefulWidget?

O: **StatelessWidget** – UI zależny tylko od konstruktorów/rodzica. **StatefulWidget** – lokalny mutowalny stan w **State**, **setState** triggeruje rebuild. Do większej logiki lepiej Cubit/Bloc/Provider niż rozbudowany **StatefulWidget**.

P: Co to jest BuildContext?

O: Uchwyt do lokalizacji widżetu w drzewie – **InheritedWidget** (**Theme**, **MediaQuery**, **Provider**), nawigacja, **showDialog**. Nie używaj **context** po **async** bez sprawdzenia **mounted/context.mounted**.

P: Do czego służą Keys?

O: Pomagają frameworkowi identyfikować widgety przy reorderingu listy, zachowaniu stanu (`ValueKey`, `ObjectKey`, `UniqueKey`, `GlobalKey`). Bez kluczy Flutter może źle zmapować elementy listy.

P: `const` konstruktor – po co?

O: Pozwala współdzielić niezmiennie widgety i ograniczać zbędne rebuildy. Tam gdzie się da – `const`.

P: Jak działa `async/await` w Dart?

O: `Future` reprezentuje wynik asynchroniczny; `async/await` to cukier nad event loopem. Ciężkie CPU lepiej przenieść do `Isolate` (`compute`), bo Dart UI to jeden wątek.

P: Stream vs Future?

O: `Future` – jedno zdarzenie. `Stream` – sekwencja zdarzeń (Firestore, websocket, tickery). `BloC` opiera się o `streamy event→state`.

P: Null safety w Dart?

O: Typy domyślnie `non-nullable`; `?` oznacza `nullable`; `!` to `assert non-null` (ostrożnie); `?.`, `??`, `late` według potrzeby. Unikam `!` bez gwarancji.

P: Różnica między `hot reload` a `hot restart`?

O: `Hot reload` wstrzykuje nowy kod zachowując stan (szybka iteracja UI). `Hot restart` resetuje stan aplikacji. Pełny restart procesu przy zmianach natywnych/pluginów.

P: Co to `InheritedWidget`?

O: Mechanizm przekazywania danych w dół drzewa bez prop-drilling. `Provider`/`Riverpod` budują na tym. `dependOnInheritedWidgetOfExactType` rejestruje zależność i rebuild przy zmianie.

P: Jak debugujesz performance UI?

O: Flutter DevTools: rebuild counts, timeline, jank. Unikam zbędnych rebuildów, ciężkiej pracy w `build`, używam `ListView.builder`, cache obrazów, `RepaintBoundary` tam gdzie ma sens.

P: `ListView` vs `ListView.builder`?

O: `ListView` buduje dzieci od razu – źle dla długich list. `builder/separated` lazy-loadują widoczne elementy.

P: Co to `Widget`, `Element`, `RenderObject`?

O: `Widget` – konfiguracja. `Element` – instancja w drzewie zarządzająca cyklem życia. `RenderObject` – layout/paint. Zrozumienie pomaga przy optymalizacji i custom renderach.

P: Material vs Cupertino?

O: Material – Android/Google design. Cupertino – iOS look. Często Material + platform adaptive (`Theme`, `Platform.isIOS`) albo własne design system klienta.

P: Jak wersjonujesz zależności w pubspec?

O: SemVer, `^` dla kompatybilnych, `pubspec.lock` commitowany w aplikacjach. Po upgrade – testy regresji i changelog breaking changes.

P: Co to package vs plugin?

O: Package – czysty Dart. Plugin – z kodem natywnym (Android/iOS) przez platform channels. Federated plugins mają osobne implementacje per platformę.

5 Zarządzanie stanem (Provider, Cubit, BloC)

P: Jakie znasz podejścia do state managementu?

O: `setState` (lokalnie), `Provider/ChangeNotifier`, `Cubit/BloC`, `Riverpod`, `GetX` (ostrożnie), `Redux-like`. W ofercie Ideo: `Cubit`, `Provider`, `BloC` – te trzy znam praktycznie (`Provider` w `Fruit Tracker`; `Cubit/BloC` z kursów i praktyki).

P: `Provider` – jak działa?

O: `ChangeNotifier` + `ChangeNotifierProvider` + `Consumer/context.watch`. Prosty, dobry na średnie apki. Łatwo przesadzić z `god-object` notifierem – lepiej dzielić odpowiedzialności.

P: `Cubit` vs `BloC`?

O: `Cubit`: wywołujesz metody, emitujesz stany – mniej boilerplate. `BloC`: eventy → mapowanie → stany, lepsze gdy chcesz jawny log eventów, złożone flow, łatwiejsze testowanie event-driven. W małym feature często `Cubit` wystarczy.

P: Kiedy wybrałbyś `BloC` zamiast `Provider`?

O: Złożona logika biznesowa, wiele źródeł eventów, potrzeba jasnych stanów UI (`loading/error/data`), testowalność, konwencja zespołu. `Provider` OK dla prostego stanu UI i DI.

P: Jak testujesz `Cubit/BloC`?

O: `bloc_test`: podajesz eventy/wywołania, oczekujesz sekwencji stanów. Mockujesz repozytoria. Osobno testujesz `reducery`/mapowanie.

P: Jak unikasz rebuildowania całego drzewa?

O: `context.select`, `BlocSelector`, dzielenie providerów, wynoszenie stałych widgetów, nie słuchanie całego modelu gdy potrzebne jest jedno pole.

P: Gdzie trzymasz side-effecty (nawigacja, `snackbary`)?

O: Nie w `build`. `Listener (BlocListener)`, `callbacki` po `Future`, albo warstwa efektów. Unikam nawigacji bezpośrednio z `Cubita` bez abstrakcji.

P: Jak wyglądał stan w Twoim `Fruit Trackerze`?

O: `Provider` do stanu spożycia, ulubionych, historii i `offline shopping planner`. Dane lokalnie w `SQLite`. Przy rozroście rozważałbym `Cubit/BloC` per feature + repozytorium nad `SQLite`.

6 REST API i komunikacja klient–serwer

P: Jak wołasz REST z Fluttera?

O: Pakiet `http` lub `dio` (`interceptory`, `timeouty`, `cancel`). Warstwa API client → repository → `Cubit/BloC`. Modele z `fromJson/toJson` (`json_serializable` / `freezed`).

P: Jak obsługujesz błędy sieciowe?

O: `Timeouty`, mapowanie statusów HTTP na wyjątki domenowe, `retry` z `backoffem` dla idempotentnych GET, stany error w UI, logowanie `Crashlytics`. Offline: `cache/kolejka`.

P: Co z autoryzacją?

O: Bearer token w interceptorze, secure storage (Keychain/Keystore via `flutter_secure_storage`), refresh token flow, wylogowanie przy 401. Nigdy tokenów w plain `SharedPreferences` na produkcję jeśli da się uniknąć.

P: REST vs GraphQL?

O: REST – zasoby, cache HTTP, prościej. GraphQL – precyzyjne query, jeden endpoint, więcej złożoności klienta. W Ideo oferta mówi REST – to znam praktycznie.

P: Jak wersjonujesz API po stronie klienta?

O: Base URL + version path/header, feature flags, graceful degradation gdy pole zniknie, nie crashować na nieznanym polu (`ignoreUnknown`).

P: Pagination – jak?

O: Offset/limit lub cursor. W UI: infinite scroll + loading footer, deduplikacja, cancel poprzedniego requestu przy szybkim scrolu.

P: Co to idempotentność i dlaczego ważna w mobile?

O: Powtórzony request nie psuje stanu (np. płatność). Przy słabej sieci mobile często retry – bez idempotencji łatwo o duplikaty.

7 Firebase

P: Z jakich usług Firebase korzystałeś?

O: W DEVEJI: backend Firebase pod aplikacje mobilne (Auth, baza, powiadomienia/crash według projektu). Potrafię ustawić konfigurację iOS/Android, reguły bezpieczeństwa i integrację z Flutter-Fire.

P: Firestore vs Realtime Database?

O: Firestore – dokumenty/kolekcje, lepsze query złożone, skalowanie. RTDB – drzewo JSON, niskie opóźnienia sync. Dziś częściej Firestore.

P: Jak zabezpieczasz dane w Firestore?

O: Security Rules po `auth.uid`, walidacja pól, unikanie client-side-only security. Minimalny privilege, testy rules.

P: Firebase Auth – przepływ?

O: Email/password, Google/Apple Sign-In, anonymous→link. Słuchasz `authStateChanges`, mapujesz na stan aplikacji, tokeny odświeżane przez SDK.

P: FCM – push?

O: Token urządzenia, tematy/segmenty, handlers foreground/background, uprawnienia iOS (APNs). Deep link z payloadu do właściwego ekranu.

P: Crashlytics i Analytics – po co?

O: Crashlytics: stacktrace, non-fatals, kontekst użytkownika. Analytics/Performance: funnele, wolne ekrany. Bez obserwowalności trudno utrzymać jakość po release.

P: Offline we Firebase?

O: Firestore ma persistence cache; trzeba projektować UX na stale data i konflikty. Przy pełnym offline-first często SQLite + sync (jak w Fruit Trackerze lokalnie).

8 UI, design i jakość wizualna

P: Jak dbasz o pixel-perfect względem designu?

O: Sprawdzam spacing/typography/kolory ze specyfikacji (Figma), używam theme (`TextTheme`, `ColorScheme`), komponenty wielokrotnego użytku, porównuję na urządzeniach o różnej gęstości.

P: Jak robisz responsywność?

O: `LayoutBuilder`, `MediaQuery`, `flexible/expanded`, czasem `adaptive` breakpoints. Unikam hardcodowanych wysokości na ślepo.

P: Animacje we Flutterze?

O: Implicit (`AnimatedContainer`), explicit (`AnimationController`), Hero, Rive/Lottie. Animacja ma wspierać hierarchię, nie hałasować.

P: Dostępność?

O: Semantics, kontrast, duże fonty (`textScaler`), cele tap $\geq 48dp$, `VoiceOver/TalkBack`. To bywa wymagane u klientów korporacyjnych.

P: Skąd bierzesz ikony/assetty?

O: Design system klienta, `flutter_svg`, asset bundle, generatory. Dbam o @2x/@3x lub wektory.

9 Natywne Android / iOS i mosty

P: Kiedy piszesz kod natywny przy Flutterze?

O: Gdy brakuje pluginu, trzeba głębokiej integracji (HealthKit-like, specjalne SDK bankowe, background modes) albo optymalizacji. `MethodChannel` / `EventChannel` / Pigeon.

P: Co to `MethodChannel`?

O: Most Flutter↔natywny: wysyłasz nazwę metody + argumenty, po stronie Swift/Kotlin obsługujesz i zwracasz wynik. Typowanie warto generować (Pigeon).

P: Masz doświadczenie natywne iOS?

O: Tak – SwiftUI: Water Reminder (`SwiftData`, lokalne powiadomienia), Fitness Tracker (`HealthKit`, `Charts`), Lotto. Znam Xcode, signing, capabilities.

P: Android natywnie?

O: Podstawy Kotlin i Android Studio; w praktyce głównie Flutter + konfiguracja Gradle/manifest/permissions. Chętnie dogram natywne mosty w projekcie.

P: Różnice iOS vs Android, o których musisz pamiętać?

O: Uprawnienia i privacy strings, push (APNs vs FCM), back gesture/navigation, lifecycle, store review guidelines, rozmiary ekranów/OEM Android.

P: Jak debugujesz problem tylko na iOS?

O: Reprodukacja na symulatorze/urządzeniu, logi Xcode, porównanie z Androidem, sprawdzenie Info.plist/capabilities, wersji OS, pluginów z issue trackerem.

10 Publikacja App Store / Google Play

P: Jak wygląda release Fluttera do sklepów?

O: Bump version/build, signing (Keystore / certificates), `flutter build appbundle/ipa`, meta-data, privacy labels, testy na real devices, staged rollout. CI (Codemagic/GitHub Actions) przyspiesza.

P: Częste powody odrzucenia App Store?

O: Brakujące privacy disclosures, niedziałający login, placeholder content, naruszenia guidelines płatności, crash na starcie, brak konta demo dla reviewera.

P: AAB vs APK?

O: Play wymaga AAB – Google generuje zoptymalizowane APK per urządzenie. APK czasem do sideload/testów.

P: Jak wersjonujesz buildy?

O: `version: x.y.z+build` w pubspec – name + number. Number musi rosnać monotonicznie w sklepach.

P: Co to TestFlight / internal testing?

O: TestFlight – beta iOS. Play: internal/closed/open testing. Zawsze waliduję krytyczne ścieżki przed produkcją.

11 Architektura, czysty kod, git

P: Jaką architekturę stosujesz we Flutterze?

O: Feature-first: presentation (UI + Cubit) / domain / data (API, DB). Dependency injection (get_it/riverpod). Unikam logiki biznesowej w widgetach.

P: Co byś poprawił w swoich wczesnych apkach Swift?

O: Water Reminder i Fitness Tracker mają dużo logiki w `ContentView` – wydzieliłbym ViewModele, serwisy (notifications/HealthKit), testy jednostkowe streaków/MET. To świadoma lekcja modularności.

P: Git workflow?

O: Feature branche, małe PR-y, sensowne commity, code review, unikam commitowania sekretów (`.env`, keystore). Preferuję rebase/squash według konwencji zespołu.

P: Czy piszesz testy?

O: Unit dla logiki/Cubitów, goldeny/widget testy dla krytycznego UI, integration na smoke path. W projektach uczelnianych bywało ich mało – w komercyjnych traktuję testy jako część Definition of Done.

P: SOLID w mobile – przykład?

O: Repository odwraca zależność UI od Dio/Firebase. Strategię dla różnych login providerów. Małe widgety z jedną odpowiedzialnością zamiast 1000-LOC ekranu.

12 Pytania o Twoje projekty

12.1 DEVEJI – Flutter komercyjnie

P: Co robiłeś w DEVEJI?

O: Mobile App Developer (Flutter), VIII–X 2025: cross-platform iOS/Android, Firebase, deployment do App Store i Google Play. To moje główne komercyjne doświadczenie Flutter pod wymagania Ideo.

P: Jakie wyzwania techniczne tam były?

O: Dopasuj do faktów: integracja Firebase, różnice platform, przygotowanie buildów, poprawki po QA/review, dbałość o UI zgodne z projektem. Nie zmyślaj szczegółów klienta objętych NDA – mów o kategoriach zadań.

12.2 Fruit & Vegetable Tracker (Flutter, prywatne)

P: Opowiedz o Fruit Trackerze.

O: Cross-platform Flutter: monitoring spożycia owoców/warzyw, skanowanie produktów modelem ML, analiza odżywcza, historia, ulubione, offline planner zakupów. Stan: Provider. Dane: SQLite lokalnie.

P: Dlaczego SQLite a nie tylko Firebase?

O: Offline-first i prywatność danych żywieniowych na urządzeniu. Szybkie lokalne query historii. Sync z chmurą można dodać później jako warstwę.

P: Jak działa skanowanie ML?

O: Model klasyfikujący produkt (on-device). Pipeline: kamera/obraz → inferencja → mapowanie na produkt/wartości odżywcze → zapis historii. Edge case'y: słabe oświetlenie, niepewność modelu – UX z potwierdzeniem użytkownika.

P: Co było najtrudniejsze?

O: Połączenie UX skanowania z wiarygodnością ML i spójnym stanem offline. Plus dopracowanie UI detali graficznych.

12.3 Water Reminder (SwiftUI)

P: Co robi Water Reminder?

O: iOS tracker nawodnienia: cel 4000 ml, szybkie dodawanie, streak, powiadomienia lokalne z akcjami, model „bojlera”, historia (SwiftData), animacje GIF.

P: SwiftData czy Core Data?

O: W kodzie jest **SwiftData**. Jeśli ktoś zapyta o README – wyjaśniam precyzyjnie: persistence przez SwiftData (**WaterProgress**, **BoilerModel**, **DailyCountModel**).

P: Jak działają powiadomienia?

O: Harmonogram w ciągu dnia, kategorie akcji (wypij / opóźnij), reset o północy. `UNUserNotificationCenterDelegate` w `AppDelegate`.

P: Co poprawisz architektonicznie?

O: Wydzielić serwis powiadomień, `ViewModel`, nie trzymać wszystkiego w jednym `ContentView`. Dodać widжет jeśli obiecamy go w opisie.

12.4 Fitness Tracker (SwiftUI)

P: Co robi Fitness Tracker?

O: Kroki z `HealthKit`, szacunek dystansu/kalorii (MET), treningi biegni, waga + wykresy (Swift Charts), profil użytkownika. `SwiftData` do persistence.

P: Jak liczysz kalorie?

O: Wzór MET: $(MET \times 3.5 \times waga \times minuty) / 200$. Osobne MET dla chodu/biegu wg prędkości. To estymacja, nie medyczny measurement.

P: Czy jest MapKit / zdjęcia postępów?

O: W aktualnym kodzie publicznym focus jest na `HealthKit` + `Charts` + profil. Jeśli ktoś porówna z marketingowym opisem – mówię uczciwie, co jest zaimplementowane, a co było w planach.

12.5 MacSpaceCleaner

P: Co to MacSpaceCleaner i czemu o tym mówisz na rozmowie Flutter?

O: Desktop (.NET 9 / Avalonia) do odzyskiwania miejsca na macOS: skan wolumenów, kategorie śmieci, heurystyki score, opcjonalne AI, twarde reguły bezpieczeństwa ścieżek. Pokazuje, że myślę warstwami (Core vs UI), o safety i UX potwierdzeń – to transferowalne do mobile.

P: Jak dbasz o bezpieczeństwo usuwania?

O: `PathSafety.CanDelete` blokuje krytyczne ścieżki systemowe; delete deepest-first; wizarz potwierdzeń; Reveal in Finder. Bez sudo threat model.

12.6 Courier Task Management (Electron, prywatne)

P: Opowiedz o systemie kurierskim.

O: Desktop Electron + Node + MySQL: planowanie zadań, tracking przesyłek, tickety, PDF raporty, panele ról (admin/manager/kurier/magazyn). Pokazuje pracę z rolami i procesami biznesowymi.

12.7 Bus Timetable (Laravel)

P: Co to za projekt?

O: Laravel 11 + MySQL: zarządzanie trasami/przystankami/czasami, role admin/kierowca, wykresy. To CRUD z politykami dostępu – **bez** realnego GPS realtime mimo że bywało tak opisane. Uczciwość > marketing.

P: Czego się nauczyłeś negatywnie?

O: Nie commitować `.env` z sekretami. Secrets w vault/CI, `.env.example` w repo.

12.8 Inne

P: Lotto Results App?

O: SwiftUI + SwiftData, zapis kuponów, WebView wyników, eksperyment scrapingu (SwiftSoup) – niedokończony względem selektorów. Główny flow: ręczny zapis + porównanie.

P: Automatic Hotspot Disconnecter?

O: Mały Python/macOS daemon: przy locku ekranu wyłącza Wi-Fi jeśli hotspot; LaunchAgent. Pokazuje automatyzację systemu, nie jest core mobile portfolio.

13 Pułapki i uczciwe odpowiedzi „co bym poprawił”

P: Widzimy, że publiczny GitHub to głównie Swift, a oferta to Flutter – jak to wyjaśnisz?

O: Komercyjny Flutter i prywatny Fruit Tracker nie są publiczne (NDA/prywatność). Publicznie pokazuję natywne iOS i inne projekty. Na rozmowie mogę przejść po architekturze Flutter, Firebase, Provider/Cubit i procesie store z DEVEJI.

P: Brak testów w repo?

O: W projektach osobistych/uczelnianych bywało słabo – wiem, że to dług. W komercyjnym Flutterze stawiam na testy Cubitów i smoke ścieżek. Mogę pokazać jak bym przetestował streak w Water Reminderze.

P: Czy nie jesteś za junior pod to stanowisko?

O: Mam komercyjny Flutter + store + Firebase, własne apki z ML/SQLite i mocne iOS. Szybko się uczę, mam szeroki kontekst. Szukam miejsca gdzie urośnie mid na realnych projektach Ideo.

14 Pytania behawioralne (STAR – szkielety)

P: Opowiedz o trudnym bugie, który naprawiłeś.

O: S: problem na jednej platformie (np. push/permissions/signing). T: znaleźć root cause. A: minimalna reprodukcja, logi natywne, porównanie Android/iOS, fix + regresja. R: stabilny release, checklista na przyszłość. *Wypełnij konkretnym przykładem z DEVEJI.*

P: Konflikt w zespole / inna opinia techniczna?

O: Słucham argumentów (perf, time-to-market, maintenance), proponuję spike/POC na czasboxed 0.5–1 dnia, decyzję zapisujemy. Nie blokuję ego.

P: Deadline vs jakość?

O: Wycinam scope (MVP), nie jakość fundamentów (crash, security). Komunikuję trade-offy wcześnie. Hotfix tylko z planem spłaty.

P: Z czego jesteś najbardziej dumny?

O: Wybierz 1: publikacja store w DEVEJI / Fruit Tracker ML+offline / MacSpaceCleaner safety architecture / Water Reminder dopracowany UX powiadomień.

15 Mini zadania kodowe / whiteboard (na wszelki wypadek)

P: Napisz model Dart z JSON.

O: `class User { final String id; final String name; User({required this.id, required this.name}); factory User.fromJson(Map<String,dynamic> j)=>User(id:j['id'], name:j['name']); }`

P: Jak zrobisz listę z API?

O: Cubit: emit loading → `repo.fetch()` → emit success/error. UI: `BlocBuilder + ListView.builder`. Pull-to-refresh wywołuje event.

P: FutureBuilder vs Cubit – kiedy co?

O: FutureBuilder OK dla prostego jednorazowego fetch w lokalnym widgetcie. Przy cache, retry, wielu ekranach, testach – Cubit/BloC.

P: Jak zrobisz debounce search?

O: Na zmianę tekstu: `timer 300ms / Rx debounce / transform eventów w BloC (debounce)`. Cancel poprzedniego HTTP.

P: Jak przekażesz dane między ekranami?

O: Argumenty routera, inherited/provider scope, result z `Navigator.pop`. Unikam singletonów-globali bez DI.

P: Napisz na tablicy różnicę `setState` vs `notifyListeners` vs `emit`.

O: `setState` – lokalny State. `notifyListeners` – `ChangeNotifier/Provider`. `emit` – Cubit nowy stan immutable, UI słucha streamu.

P: Co wypisze kod z race na context po `await`?

O: Ryzyko użycia context po `unmount` – crash/warning. Fix: `if (!context.mounted) return;` przed nawigacją/snackbarem.

P: Zadanie: cache obrazków.

O: `cached_network_image` lub własny: memory + disk LRU, placeholder/error widget, poprawne key URL.

16 Android / iOS – pytania pogłębiające

P: Co to AppDelegate / MainActivity w kontekście Fluttera?

O: Punkty wejścia natywne hostujące FlutterEngine. Tu rejestrujesz pluginy, kanały, powiadomienia, deep linki.

P: ProGuard / R8 / obfuscation?

O: Na Androidzie minifikacja w release; trzeba keep rules dla refleksji/JSON. Na iOS bitcode już historyczne – ważniejsze stripping i App Size.

P: Background fetch / restrictions?

O: OS mocno ogranicza tło. Trzeba właściwych modes/permissions i realistycznych oczekiwań biznesu. Często push + przy najbliższym foreground sync.

P: Deep links / Universal Links / App Links?

O: Konfiguracja associated domains / Digital Asset Links, routowanie w Flutter (`go_router`), testy z cold/warm start.

17 Bazy lokalne i offline

P: SharedPreferences vs Hive vs SQLite vs Drift/Isar?

O: Prefs – małe key-value. Hive/Isar – szybkie NoSQL na urządzeniu. SQLite/Drift – relacyjne, query, dobre do historii spożycia (Fruit Tracker).

P: Migracje schematu?

O: Wersjonowanie DB, migracje w górę, backup przed migracją, test na świeżej i starej instalacji. Nigdy nie dropować produkcji bez ścieżki.

P: Jak projektujesz offline-first?

O: Lokalne źródło prawdy, kolejka mutacji, sync z resolucją konfliktów (last-write-wins lub merge), UX wskaźnik sync.

18 Bezpieczeństwo mobile

P: Jak chronisz sekrety w aplikacji?

O: Brak tajnych kluczy API w binarkach jeśli da się uniknąć – backend proxy. Certificate pinning rozważnie. Secure storage na tokeny. Obfuscation to tylko utrudnienie, nie security boundary.

P: OWASP Mobile – przykłady?

O: Niezabezpieczone storage, słaba TLS, niezaufane WebView, debug residual w release, nadmierne uprawnienia.

P: ATS / cleartext traffic?

O: iOS ATS blokuje HTTP; Android cleartext domyślnie off w nowszych. Tylko świadome wyjątki.

19 Wydajność i stabilność

P: Skąd biorą się janki UI?

O: Ciężka praca na UI isolate, zbędne rebuildy, ogromne obrazy bez cache, synchroniczne IO, shader compilation (Impeller poprawia).

P: Jak zmniejszysz rozmiar apki?

O: Split ABI/AAB, kompresja assetów, usuwanie dead code, deferred components rzadziej, analiza `flutter build size`.

P: Memory leaks we Flutterze?

O: Niezwolnione controllery/streamy, GlobalKey nadużycia, cache bez limitu, listenery. Dispose w `State.dispose` / close Cubit.

20 Proces, Agile, współpraca z designem

P: Jak czytasz taska w Jira?

O: Cel, AC, design, edge cases, zależności API, estymacja, ryzyka. Dopytam zanim wejść w kod na 2 dni w złą stronę.

P: Code review – czego oczekujesz?

O: Feedback o architekturze, edge cases, nazewnictwie, testach. Sam reviewuję pod kątem czytelności i regresji, nie gustu formatowania jeśli jest linter.

P: Definition of Done?

O: AC spełnione, testy, brak crash na happy path, UI zgodne, obsługa błędów, wersje i18n jeśli wymagane, dokumentacja jeśli API publiczne.

21 Pytania o Ideo i technologię firmy

P: Co wiesz o Ideo?

O: Software house ~250 osób, Rzeszów (+ Warszawa), dedykowane systemy: intranet, workflow, e-learning, lojalty, e-commerce, B2B. Stack m.in. .NET, PHP/Laravel, TS/Node, Vue/React, Swift, Flutter. Kariera: kariera.ideo.pl.

P: Dlaczego Flutter u Was, nie tylko natywnie?

O: *Pytanie do nich* – spodziewana odpowiedź: time-to-market, wspólny kod, zespół mobilny w rozbudowie, klienci multi-platform.

22 Twoje pytania do Ideo (zadaj 3–5)

1. Jaki state management jest standardem w zespole mobilnym (BloC vs Cubit vs Provider)?
2. Jak wygląda code review i CI/CD release do sklepów?
3. Nad jakimi typami klientów/projektów pracowałby zespół Flutter teraz?
4. Czy junior/mid dostaje mentora i jak mierzony jest okres próbny?
5. Jaki udział pracy natywnej Swift/Kotlin vs czysty Flutter?
6. Hybryda: ile dni w biurze na Nad Przyrwać na starcie?
7. Czy są testy automatyczne / golden tests w projektach?
8. Jakie są kolejne etapy rekrutacji po dzisiejszym spotkaniu?

23 Checklista dzień przed / rano

- CV i portfolio zgodne z tym, co mówisz (SwiftData nie Core Data; bez zmyślania MapKit jeśli nie ma w kodzie).
- 2–3 zdania o DEVEJI, Fruit Tracker, Water Reminder, MacSpaceCleaner.
- Przypomnij Cubit vs BloC vs Provider na 1 minutę.
- Firebase Auth + Firestore rules w 1 minutę.

- Pytania do nich wypisane.
- Dojazd ul. Nad Przyrwą 13, 10:00, ≤ 45 min.
- Laptop opcjonalnie – raczej dyskusja, nie live IDE; ale miej repo/readme pod ręką.

Powodzenia na rozmowie 11.09.2026 – mów konkretami z projektów, uczciwie o lukach, z energią wokół Fluttera + native.